

flisp 语言参考手册

一个基于极小核心的 Lisp 方言解释器，使用 tagged pointer、深拷贝闭包、goto 尾调用优化。
内置 27 个原语 + 约 20 个系统库函数/宏。

目录

1. [词法语法](#)

- [1.1 注释](#)
- [1.2 数值字面量](#)
- [1.3 符号](#)
- [1.4 列表](#)
- [1.5 引号语法糖](#)

2. [类型系统](#)

- [2.1 类型概览](#)
- [2.2 布尔值](#)
- [2.3 相等判定](#)

3. [特殊形式](#)

- [3.1 quote —— 阻止求值](#)
- [3.2 fn —— 创建函数 \(λ 表达式\)](#)
- [3.3 macro —— 创建宏](#)
- [3.4 def —— 定义全局变量](#)
- [3.5 cond —— 条件分支](#)
- [3.6 do —— 顺序执行](#)
- [3.7 and —— 短路与](#)
- [3.8 or —— 短路或](#)

4. [内置函数](#)

- [4.1 算术](#)
- [4.2 比较](#)
- [4.3 逻辑](#)
- [4.4 列表操作](#)
- [4.5 类型谓词](#)
- [4.6 元编程](#)
- [4.7 输出](#)

5. [函数与闭包](#)

- [5.1 固定参数](#)
- [5.2 变参函数](#)
- [5.3 纯变参 \(裸符号\)](#)
- [5.4 闭包捕获](#)
- [5.5 尾递归](#)

6. [宏](#)

- [6.1 定义宏](#)
- [6.2 宏展开过程](#)
- [6.3 宏与函数的本质区别](#)

7. [系统库 \(system.lisp\)](#)

- [7.1 语法宏](#)

- [7.2 列表工具](#)
 - [7.3 高阶函数](#)
 - [7.4 准引用 \(quasiquote\)](#)
 - 8. [准引用详解](#)
 - [8.1 读者转换](#)
 - [8.2 宏展开](#)
 - [8.3 展开示例](#)
 - [8.4 拼接语法 ,,@](#)
 - 9. [eval 与 apply](#)
 - 10. [已知限制](#)
 - 11. [内置函数速查表](#)
 - 12. [系统库速查表](#)
-

1. 词法语法

1.1 注释

使用分号 `;`：从 `;` 到行尾全部忽略。

```
; 这是注释  
(def x 42) ; 行末注释
```

无块注释语法。

1.2 数值字面量

只支持十进制整数：

```
42  
0  
1000000
```

限制：

- 不支持负数直接量：`-5` 会被解析为符号 `-5`，必须用 `(- 5)` 表示负数
- 不支持浮点数：`1.2` 会静默解析为错误整数
- 不支持其他进制

1.3 符号

除空白、`(`、`)`、`;`、`'`、```、`,` 外的任意字符均可构成符号名。符号被 intern 在全局符号表中。

```
hello  
+  
a-b-c?  
*  
cons
```

几个特殊符号在系统中有预定义含义：

符号	用途
<code>nil</code>	假值、空
<code>#t</code>	真值
<code>&</code>	变参标记（函数参数中）
<code>else</code>	cond 兜底分支（由 <code>system.lisp</code> 定义为 <code>#t</code> ）
<code>fn</code>	函数类型标签
<code>macro</code>	宏类型标签
<code>quote</code> 、 <code>quasiquote</code> 、 <code>unquote</code> 、 <code>unquote-splicing</code>	引号系统

1.4 列表

```
(1 2 3)      ; 三个元素的列表
()           ; 空列表（等同于 '()）
(1 (2 3) 4)  ; 嵌套列表
```

`head` 取首元素，`tail` 取剩余列表，`cons` 构造列表。

1.5 引号语法糖

读者（reader）在解析时做一层语法糖替换，不涉及求值语义：

输入	读者输出
<code>'expr</code>	<code>(quote expr)</code>
<code>`expr</code>	<code>(quasiquote expr)</code>
<code>,expr</code>	<code>(unquote expr)</code>
<code>,@expr</code>	<code>(unquote-splicing expr)</code>

```
'x      ; → (quote x)
'(a b c) ; → (quote (a b c))
`(a ,b)  ; → (quasiquote (a (unquote b)))
```

2. 类型系统

2.1 类型概览

所有值以 tagged pointer 表示（`uintptr_t`），低 2 位编码类型：

类型	谓词	表示	说明
Number	<code>number?</code>	值左移 2 位，零堆分配	32 位整数
List	<code>list?</code>	指向 <code>{head, tail}</code> 的指针	cons cell 链
Symbol	<code>symbol?</code>	指向符号表条目的指针	interned
Builtin	<code>builtin?</code>	指向内置函数结构体的指针	含 code 枚举

```
(number? 42)      ; → #t
(symbol? 'hello)  ; → #t
(list? '(1 2 3))  ; → #t
(builtin? +)      ; → #t
```

2.2 布尔值

值	真值
<code>nil</code> (符号)	假
<code>'()</code> (空列表)	假
其他全部 (含 <code>0</code> 、 <code>#t</code> 、非空列表等)	真

```
(if 0 'true 'false) ; → true (0 不是 nil 也不是空列表，视为真)
(if '() 'true 'false) ; → false
(if nil 'true 'false) ; → false
```

2.3 相等判定

= 执行结构递归比较：

```
(= 3 3)           ; → #t 数字按值比较
(= 'a 'a)         ; → #t 符号按 identity 比较
(= '(1 2 3) '(1 2 3)) ; → #t 列表递归比较各元素
(= '(1 2) '(1 2 3)) ; → nil 长度不同
(= '() '())       ; → #t
```

3. 特殊形式

特殊形式的参数不自动求值，由自身控制求值时机。

3.1 `quote` —— 阻止求值

```
(quote expr) → 返回 expr 本身
'expr       → 同上（语法糖）
```

```
(print (quote a)) ; 输出: a (符号, 不查找变量)
(print '(1 2 3)) ; 输出: (1 2 3) (列表, 不会尝试调用 1)
```

3.2 fn —— 创建函数 (λ 表达式)

```
(fn (params) body...)  
(fn bare-symbol body)    ; 变参形式
```

创建一个闭包，返回值为 `(FN (params) copied-body)`。

```
; 基本用法  
(def double (fn (x) (* x 2)))  
(double 5)                ; → 10  
  
; 多参数  
(def add (fn (a b) (+ a b)))  
(add 3 4)                  ; → 7  
  
; 多表达式函数体 (隐含 do)  
(def foo (fn (x)  
  (print 'calc)  
  (* x 2)))                ; 返回 (* x 2) 的值  
  
; 匿名函数直接调用  
((fn (x) (* x x)) 6)       ; → 36
```

3.3 macro —— 创建宏

```
(macro (params) body...)
```

创建宏，返回值为 `(MACRO (params) copied-body)`。宏的参数**不求值**，宏体返回的 S-表达式会被**再次求值**。

```
; 将表达式执行两次  
(def twice (macro (expr)  
  (list 'do expr expr)))  
  
(twice (print 42))        ; 输出 42 两次  
  
; 自定义条件  
(def my-if (macro (c t e)  
  (list 'cond (list c t) (list 'else e))))  
  
(my-if (< 1 2) 'yes 'no)  ; → yes
```

3.4 def —— 定义全局变量

```
(def name value)
```

定义或重绑定全局符号。**始终操作全局符号表**，不进入词法环境栈。

```
(def x 10)                ; 定义 x = 10  
(print x)                 ; → 10  
(def x (+ x 1))           ; 重新赋值  
(print x)                 ; → 11
```

3.5 cond —— 条件分支

```
(cond (test1 expr1)
      (test2 expr2)
      ...
      (else exprN))
```

从上到下依次求值条件，第一个为真的条件对应的表达式被求值并返回。`else` 是约定俗成的兜底标记（其值为 `#t`）。

```
(cond ((< 1 2) 'yes) (else 'no)) ; → yes
(cond (nil 'first) (#t 'second)) ; → second
(cond (nil 'a) (nil 'b) (else 'c)); → c
```

3.6 do —— 顺序执行

```
(do expr1 expr2 ... exprN)
```

依次求值所有表达式，返回最后一个的值。

```
(do 1 2 3) ; → 3
(do (print 'a) (print 'b) 42) ; 输出 a 然后 b，返回 42
```

3.7 and —— 短路与

```
(and expr1 expr2 ...)
```

从左到右求值，遇到假值（`nil` 或 `'()`）停止并返回该假值；否则返回最后一个表达式的值。

```
(and 1 2 3) ; → 3
(and 1 nil 3) ; → nil
(and 0 1 2) ; → 2 (0 是真值)
```

3.8 or —— 短路或

```
(or expr1 expr2 ...)
```

从左到右求值，遇到真值停止并返回该值；否则返回最后一个值。

```
(or nil 2 3) ; → 2
(or nil nil) ; → nil
(or 0 1) ; → 0 (0 是真值，短路)
```

4. 内置函数

内置函数的参数全部自动求值后再传入。

4.1 算术

所有算术函数均支持变参（至少 1 个参数），运算顺序从左到右。

```
(+ 1 2)           ; → 3    加法
(+ 1 2 3 4 5)     ; → 15
(- 10 3)          ; → 7    减法
(- 5)             ; → -5   单参数取负
(- 10 3 2)        ; → 5    10-3-2
(* 2 3)           ; → 6    乘法
(* 2 3 4)         ; → 24
(/ 10 2)          ; → 5    整数除法
(/ 10 3)          ; → 3    向下取整
(/ 1 0)           ; 错误: Division by zero
```

4.2 比较

精确 2 个参数，返回 `#t` 或 `nil`。

```
(< 2 3)           ; → #t
(< 5 2)           ; → nil
(> 3 2)           ; → #t
(= 3 3)           ; → #t   递归结构相等
(= '(1 2) '(1 3)) ; → nil
```

4.3 逻辑

```
(not nil)         ; → #t
(not '#t)         ; → nil
(not 0)           ; → nil (0 是真值)
(not '())         ; → nil (空列表是假值, not 后为 #t?...)
```

注意: `(not '())` → `#t`, 因为 `'()` 是假值。

4.4 列表操作

```
(cons 1 '(2 3))    ; → (1 2 3) 构造列表
(cons 1 '())       ; → (1)
(head '(a b c))    ; → a      取首元素
(head (cons 1 2))  ; 错误: tail must be a list
(tail '(a b c))    ; → (b c)  取剩余
(tail '(a))        ; → ()
(head '())         ; 错误: Trying to take head/tail of empty list
```

4.5 类型谓词

```
(number? 42)      ; → #t
(number? 'x)      ; → nil
(symbol? 'hello)  ; → #t
(symbol? 42)      ; → nil
(list? '(1 2))    ; → #t
(list? '())       ; → #t 空列表也是列表
(builtin? +)      ; → #t
(builtin? 42)     ; → nil
```

4.6 元编程

```
(eval '(+ 1 2))    ; → 3 动态求值
(eval (list '+ 1 2)) ; → 3 构造代码再执行

(apply + '(10 20 30)) ; → 60 以列表形式传参
(apply * '(2 3 4))    ; → 24
(apply (fn (x y) (+ x y)) '(10 20)) ; → 30
```

4.7 输出

```
(print 42)          ; 输出 42, 返回 nil
(print 'hello)      ; 输出 hello
(print '(a b c))    ; 输出 (a b c)
(print +)           ; 输出 #<builtin + >
(print '())         ; 输出 ()
(print 1 2 3)       ; 分别输出三行: 1 2 3
```

5. 函数与闭包

5.1 固定参数

参数列表用括号括起，逐一绑定：

```
(def add (fn (a b) (+ a b)))
(add 3 4) ; a=3, b=4 → 7
```

多表达式函数体自动按 `do` 求值，返回最后一个表达式：

```
(def foo (fn (x)
  (print 'computing)
  (* x 2)))
(foo 5) ; 输出 computing, 返回 10
```

5.2 变参函数

在参数列表中使用 `&` 标记，`&` 后的符号绑定剩余所有参数组成的列表：


```
(def f (fn (x & y) y))
(f 1 2 3)           ; x=1, y=(2 3)

(def sum-all (fn (& xs) (fold + 0 xs)))
(sum-all 1 2 3 4)   ; xs=(1 2 3 4) → 10
```

5.3 纯变参（裸符号）

参数列表直接写一个符号（不带括号），所有参数打包为列表绑定到该符号：

```
(def list (fn args args))
(list 1 2 3) ; → (1 2 3)
```

这是 `system.lisp` 中 `list` 函数的定义方式。

5.4 闭包捕获

`fn` 创建闭包时，`copy-body` 深拷贝函数体，将当前环境中绑定的自由变量替换为实际值（**深拷贝捕获语义**）：

```
(def make-adder (fn (x)
  (fn (y) (+ x y))))

(def add5 (make-adder 5))
(add5 10) ; → 15 （闭包体内 x 已被替换为 5）

; 验证：局部绑定不影响已创建的闭包
(def make-counters (fn ()
  (let ((c 0))
    (list (fn () (do (def c (+ c 1)) c))
          (fn () (do (def c (- c 1)) c)))))))
```

5.5 尾递归

解释器通过 `goto tail_eval` 实现尾调用优化，尾递归函数不消耗调用栈：

```
; 尾递归：安全的 deep recursion
(def sum-to (fn (n acc)
  (if (= n 0) acc
      (sum-to (- n 1) (+ acc n)))))

(sum-to 10000 0) ; → 50005000, 不会栈溢出

; 非尾递归：会栈溢出
(def sum-bad (fn (n)
  (if (= n 0) 0
      (+ n (sum-bad (- n 1))))))
```

规则：函数体最后一个表达式若为递归调用，则自动优化。

6. 宏

6.1 定义宏

```
(defmacro name (params & body)
  ...)

; 等价于:
(def name (macro (params) (splice-body body)))
```

`defmacro` 由 `system.lsp` 提供，是定义宏的标准方式。

```
(defmacro unless (c & b)
  (list 'if (list 'not c) (cons 'do b)))

(unless nil (print 'yes)) ; 执行 print
```

6.2 宏展开过程

1. 宏的实参**不求值**（保持为原始 S-表达式）
2. 宏体执行，返回一个新的 S-表达式（宏展开结果）
3. 展开结果被**再次求值**

```
(unless #t (print 'hello))
↓ 宏展开
(if (not #t) (do (print 'hello)))
↓ 求值
(cond ...) → nil (不打印)
```

6.3 宏与函数的本质区别

特性	函数 (fn)	宏 (macro)
参数求值	是	否
返回值	最终值	S-表达式（再求值）
展开链	无	展开结果继续 eval
tail_macro	+1	+2

7. 系统库 (system.lsp)

以下定义由 `system.lsp` 在启动时加载，非内置但随解释器提供。

7.1 语法宏

```

(defun name (params) body...)
; 展开为: (def name (fn params body...))
; 自动处理多表达式函数体 (splice-body)

(defmacro name (params & body) body...)
; 展开为: (def name (macro params (splice-body body)))

(if c t e)           ; → (cond (c t) (else e))
(if c t)              ; → (cond (c t))

(let ((x 1) (y 2))
  body...)
; 展开为: ((fn (x y) body...) 1 2)

```

7.2 列表工具

```

(list a b c)           ; → (a b c) 构造列表
(null? lst)            ; → #t / nil 判空
(atom? x)              ; → #t / nil 是否为原子 (非列表)
(fst lst)              ; → 等价于 head
(snd lst)              ; → 等价于 (head (tail lst))
(append l1 l2)         ; → 拼接两个列表
(zip '(a b) '(1 2))    ; → ((a 1) (b 2)) 转置

```

7.3 高阶函数

```

(map f '(1 2 3))       ; → ((f 1) (f 2) (f 3))
(fold f init '(a b c)) ; → (f (f (f init a) b) c) 左折叠
(foldl f '(a b c))     ; → (fold f a '(b c)) 首元素作初值
(any? p '(a b c))      ; → 任一元素满足 p?

```

7.4 准引用 (quasiquote)

```

`(1 ,(+ 1 1) 3)        ; → (1 2 3)
`(0 ,@(1 2 3) 4)       ; → (0 1 2 3 4)

```

详见下一节。

8. 准引用详解

准引用是从模板生成代码的核心工具。整个实现分为读者转换 + 宏展开两步。

8.1 读者转换

读者把语法糖转为标准形式 (纯语法替换, 无语义) :

```

(a ,(+ 1 2) c)
;; 读者输出:
(quasiquote (a (unquote (+ 1 2)) c))

```

8.2 宏展开

`quasiquote` 是在 `system.lisp` 中定义的宏，委托给 `qq-expand`：

```
(qq-expand (a (unquote (+ 1 2)) c))
→ (append (qq-expand-list a) (qq-expand ((unquote (+ 1 2)) c)))
→ (append (quote (a)) (qq-expand ((unquote (+ 1 2)) c)))
→ (append (quote (a)) (append (qq-expand-list (unquote (+ 1 2))) (qq-expand
(c))))
→ (append (quote (a)) (append (list (+ 1 2)) (quote (c))))
→ (a 3 c)      ; 求值后
```

8.3 展开规则

模式	展开
<code>`atom</code>	<code>(quote atom)</code>
<code>`()</code>	<code>(quote ())</code>
<code>`(,x)</code>	<code>(list x)</code>
<code>`(a b)</code>	<code>(append (list 'a) (list 'b))</code>
<code>`(a ,@x)</code>	<code>(append (list 'a) x)</code>
<code>`(a ` ,b c)</code>	嵌套逐层展开

8.4 拼接语法 `,@`

```
(def items '(2 3))

`(1 ,items 4)      ; → (1 (2 3) 4)    items 整体嵌入
`(1 ,@items 4)     ; → (1 2 3 4)     items 内容展平拼接

; 等价展开：
`(1 ,@items 4)
;; → (append (list '1) items (list '4))
```

9. eval 与 apply

```
(eval expr)        ; 显式求值表达式
(apply fn args)    ; 以列表 args 为参数调用 fn
```

```
; eval 动态求值
(eval '(+ 1 2))           ; → 3
(eval (list '+ 1 2))      ; → 3

; apply 列表传参
(apply + '(10 20 30))     ; → 60
(apply (fn (x y) (+ x y)) '(10 20)) ; → 30

; 与变参配合实现"拼接调用"
(def f (fn (x & y) (cons x y)))
(apply f '(1 2 3))       ; → (1 2 3)
```

10. 已知限制

限制	说明	替代方案
无负数字面量	-5 是符号	用 (- 5)
无浮点数	1.2 被错误解析	仅支持整数
无字符串	无双引号字符串	用符号或数字
无字符字面量	无 #\a 语法	—
无数组/向量	仅列表一种集合	—
无块注释	仅 ; 行注释	—
'() 必须加引号	裸 () 读入会报括号错误	用 '()
数字后需有空白或括号	文件末尾数字无换行可能死循环	—

11. 内置函数速查表

名称	类型	参数	说明
<code>quote</code>	特殊	1	返回参数本身，不求值
<code>fn</code>	特殊	变参	创建闭包
<code>macro</code>	特殊	变参	创建宏
<code>def</code>	特殊	2	定义/重绑定全局变量
<code>cond</code>	特殊	变参	多分支条件
<code>do</code>	特殊	变参	顺序执行，返回末值
<code>and</code>	特殊	变参	短路逻辑与
<code>or</code>	特殊	变参	短路逻辑或
<code>+</code>	函数	≥ 1	加法
<code>-</code>	函数	≥ 1	减法 / 取负
<code>*</code>	函数	≥ 1	乘法
<code>/</code>	函数	≥ 1	整数除法
<code><</code>	函数	2	小于
<code>></code>	函数	2	大于
<code>=</code>	函数	2	结构递归相等
<code>not</code>	函数	1	逻辑非
<code>cons</code>	函数	2	构造 cons cell
<code>head</code>	函数	1	取首元素
<code>tail</code>	函数	1	取剩余列表
<code>number?</code>	函数	1	数字判定
<code>symbol?</code>	函数	1	符号判定
<code>list?</code>	函数	1	列表判定
<code>builtin?</code>	函数	1	内置函数判定
<code>eval</code>	函数	1	显式求值
<code>apply</code>	函数	2	以列表传参调用
<code>print</code>	函数	变参	打印输出

12. 系统库速查表

名称	类型	说明
<code>defun</code>	宏	<code>(defun name args body...)</code> 定义函数
<code>defmacro</code>	宏	<code>(defmacro name args body...)</code> 定义宏
<code>if</code>	宏	<code>(if c t e)</code> 条件判断
<code>let</code>	宏	<code>(let ((x v) ...) body...)</code> 局部绑定
<code>list</code>	函数	<code>(list a b ...)</code> 构造列表
<code>null?</code>	函数	<code>(null? lst)</code> 判空
<code>atom?</code>	函数	<code>(atom? x)</code> 判原子
<code>fst</code>	函数	首元素 (head 别名)
<code>snd</code>	函数	第二个元素
<code>map</code>	函数	<code>(map f lst)</code> 映射
<code>fold</code>	函数	<code>(fold f init lst)</code> 左折叠
<code>fold1</code>	函数	首元素作初值的左折叠
<code>any?</code>	函数	任一满足谓词
<code>append</code>	函数	列表拼接
<code>zip</code>	函数	多列表转置
<code>quasiquote</code>	宏	准引用 (<code>`</code> 的底层)
<code>qq-expand</code>	函数	<code>quasiquote</code> 展开 (一阶段)
<code>qq-expand-list</code>	函数	<code>quasiquote</code> 列表元素展开
<code>splice-body</code>	函数	多表达式体自动包裹 <code>do</code>
<code>else</code>	值	<code>#t</code> , <code>cond</code> 兜底标记

本文档对应 `flisp` 解释器实现, 基于 `lisp_eval.c`、`lisp.h`、`system.lisp` 等源码。